Student Record and Academic Management System

Complete Answer / Project Documentation

Based on the uploaded assignment: Data Organization using Python

The assignment requires a Student Record Management System using Python collections and a structured development workflow: problem understanding, algorithms, flowcharts, pseudocode, implementation, testing, and documentation.

1. Problem Analysis

Objective

Develop a Python-based Student Record Management System that can store, organize, update, search, delete, and retrieve academic information efficiently using built-in data structures.

Information to be stored

• Roll number and registration number

• Student name

• Department

• Subjects

• Marks

• Attendance

• Date of birth

• Address and email/contact information

• Student clubs

Inputs

Student details, subject names, marks, attendance, department and menu choices for operations such as add, search, update, delete and reports.

Outputs

Student profiles, search results, updated/deleted records, average marks, highest scorer, department-wise listing and student count.

Relationships

A student profile is represented by a dictionary. Lists store subjects, marks and attendance. Tuples store fixed information such as roll/registration numbers and date of birth. Sets store unique departments, subjects and clubs.

2. Data Structure Selection

Data structure	Use in system
String	Name, department, address, email
List	Student records, subjects, marks, attendance
Tuple	Roll number, registration number, date of birth
Set	Unique departments, subjects and clubs
Dictionary	Complete student profile using key-value pairs

Why integration is useful

Different collections have different strengths. Dictionaries provide named fields, lists hold changeable ordered data, tuples protect fixed values, sets keep unique values, and strings represent text. Combining them gives a practical student management system.

3. Algorithms

Add Student

1. Input roll number.
2. Check whether the roll number already exists.
3. Input student profile details.
4. Store subjects, marks and attendance in lists.
5. Store fixed identifiers in tuples.
6. Add the profile dictionary to the student list.
7. Display confirmation.

Search Student

1. Input roll number.
2. Traverse the student records.
3. Compare the roll number with each record.
4. Display the matching profile.
5. If no match exists, display 'Student not found'.

Update Record

1. Input roll number.
2. Search for the student.
3. If found, input the new information.
4. Update the selected dictionary fields.
5. Display confirmation.

Delete Record

1. Input roll number.
2. Search for the student index.
3. If found, remove that record.
4. Display confirmation.

Calculate Average

1. Select a student.
2. Read the marks list.
3. Add all marks.
4. Divide total by number of subjects.
5. Display the average.

Highest Scorer

1. Check all student records.
2. Calculate each student's average.
3. Keep the highest average and corresponding student.
4. Display the student.

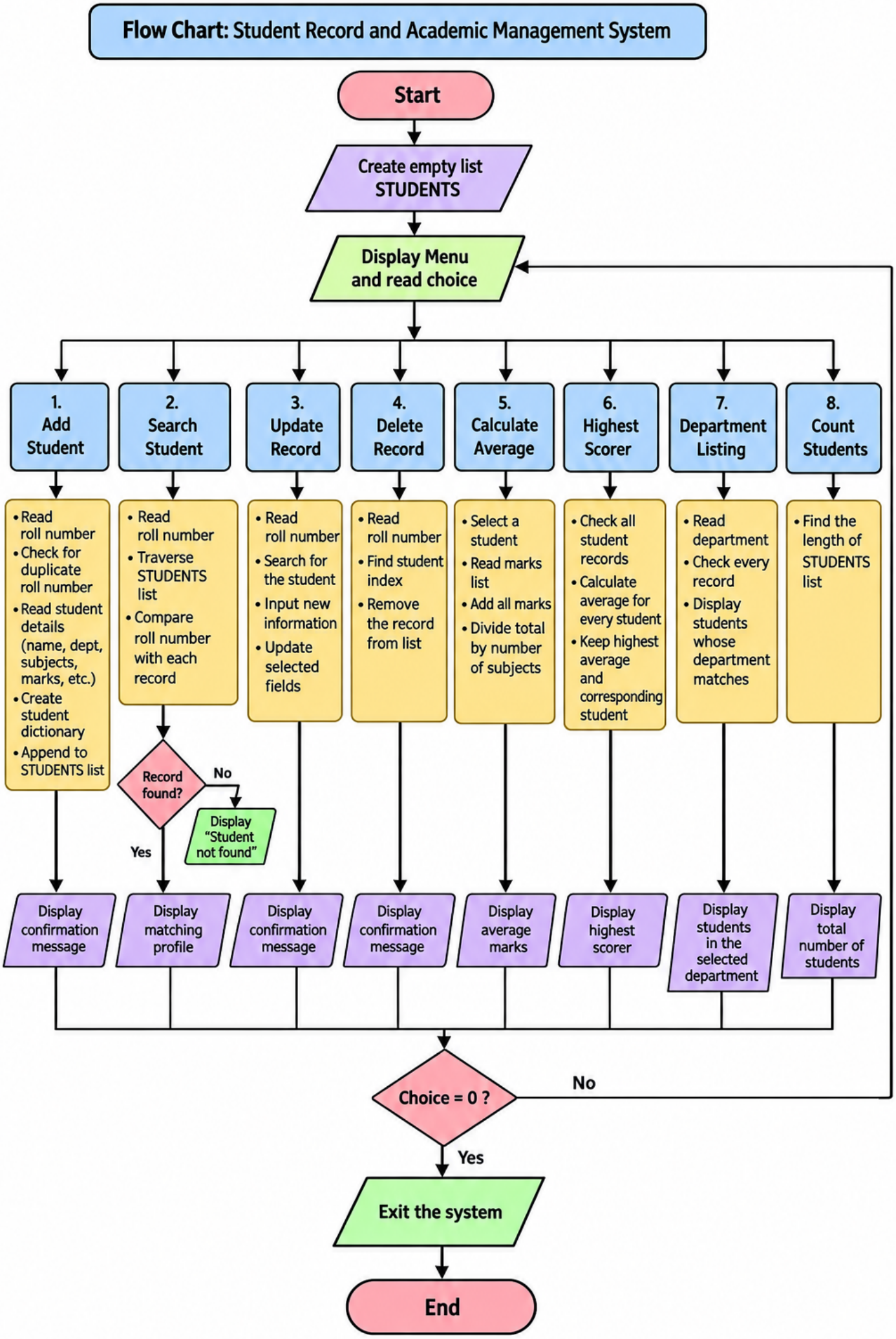
Department Listing

1. Input department.
2. Check every record.

3. Display students whose department matches.

Count Students

1. Find the length of the student record list.
2. Display the count.


Flow Chart: Student Record and Academic Management System
Start
Create empty list STUDENTS
Display Menu and read choice
1. Add Student
2. Search Student
3. Update Record
4. Delete Record
5. Calculate Average
6. Highest Scorer
7. Department Listing
8. Count Students
• Read roll number
• Check for duplicate roll number
• Read student details (name, dept, subjects, marks, etc.)
• Create student dictionary
• Append to STUDENTS list
• Read roll number
• Traverse STUDENTS list
• Compare roll number with each record
• Read roll number
• Search for the student
• Input new information
• Update selected fields
• Read roll number
• Find student index
• Remove the record from list
• Select a student
• Read marks list
• Add all marks
• Divide total by number of subjects
• Check all student records
• Calculate average for every student
• Keep highest average and corresponding student
• Read department
• Check every record
• Display students whose department matches
• Find the length of STUDENTS list
Record found?
No
Display "Student not found"
Yes
Display confirmation message
Display matching profile
Display confirmation message
Display confirmation message
Display average marks
Display highest scorer
Display students in the selected department
Display total number of students
Choice = 0 ?
No
Yes
Exit the system
End

4. Pseudocode

```
START
Create empty list STUDENTS

REPEAT
    Display menu
    Read choice

    IF choice = 1
        Read student details
        Check duplicate roll number
        Create student dictionary
        Append dictionary to STUDENTS

    ELSE IF choice = 2
        Read roll number
        Search STUDENTS
        Display matching record

    ELSE IF choice = 3
        Read roll number
        Search record
        Update required fields

    ELSE IF choice = 4
        Read roll number
        Search and delete record

    ELSE IF choice = 5
        Read marks
        Calculate sum / number of marks
        Display average

    ELSE IF choice = 6
        Calculate average for every student
        Display highest scorer

    ELSE IF choice = 7
        Read department
        Display matching students

    ELSE IF choice = 8
        Display number of students

    ELSE IF choice = 9
        Display all records

    ELSE IF choice = 0
        STOP

    ELSE
        Display invalid choice
UNTIL choice = 0
END
```

5. Flowchart Logic

Main flow: Start → Display Menu → Read Choice → Perform selected operation → Display result → Return to Menu. If choice is 0, stop the program.

Add Student flow: Start → Enter roll number → Duplicate? → Yes: show error → No: enter details → create dictionary → append record → confirmation → menu.

Search/Update/Delete flow: Enter roll number → Search records → Found? → Yes: perform requested operation → No: show 'not found' → menu.

6. Python Source Code

```
students = []

def find_student(roll):
    for student in students:
        if student["roll_no"] == roll:
            return student
    return None

def add_student():
    roll = input("Roll number: ").strip()
    if find_student(roll):
        print("Duplicate roll number.")
        return

    reg = input("Registration number: ").strip()
    name = input("Name: ").strip()
    dept = input("Department: ").strip()
    dob = input("Date of birth: ").strip()
    address = input("Address: ").strip()
    email = input("Email: ").strip()

    subjects = input("Subjects (comma separated): ").split(",")
    subjects = [s.strip() for s in subjects if s.strip()]
    marks = [float(x) for x in input("Marks (comma separated): ").split(",")]
    attendance = [float(x) for x in input("Attendance % (comma separated): ").split(",")]
    clubs = set(x.strip() for x in input("Clubs (comma separated): ").split(",")
                if x.strip())

    student = {
        "identity": (roll, reg, dob),       # tuple
        "roll_no": roll,
        "name": name,
        "department": dept,
        "subjects": subjects,               # list
        "marks": marks,
        "attendance": attendance,
        "contact": {"address": address, "email": email},
        "clubs": clubs                     # set
    }
    students.append(student)
    print("Student added successfully.")

def search_student():
    roll = input("Enter roll number: ").strip()
    s = find_student(roll)
    print(s if s else "Student not found.")

def update_student():
    roll = input("Enter roll number: ").strip()
    s = find_student(roll)
    if not s:
        print("Student not found.")
        return
    s["name"] = input(f"Name [{s['name']}]: ").strip() or s["name"]
    s["department"] = input(
        f"Department [{s['department']}]: ").strip() or s["department"]
    print("Record updated.")

def delete_student():
    roll = input("Enter roll number: ").strip()
    for i, s in enumerate(students):
        if s["roll_no"] == roll:
            students.pop(i)
            print("Record deleted.")
            return
    print("Student not found.")

def average(s):
    return sum(s["marks"]) / len(s["marks"]) if s["marks"] else 0

def show_average():
```

```
    roll = input("Enter roll number: ").strip()
    s = find_student(roll)
    if s:
        print("Average:", round(average(s), 2))
    else:
        print("Student not found.")

def highest_scorer():
    if not students:
        print("No records.")
        return
    s = max(students, key=average)
    print("Highest scorer:", s["name"], "Average:", round(average(s), 2))

def list_by_department():
    dept = input("Enter department: ").strip()
    found = [s for s in students
             if s["department"].lower() == dept.lower()]
    if found:
        for s in found:
            print(s["roll_no"], s["name"])
    else:
        print("No students found.")

def display_all():
    if not students:
        print("No records.")
    for s in students:
        print(s)

def main():
    while True:
        print("\n1 Add  2 Search  3 Update  4 Delete")
        print("5 Average  6 Highest  7 Department")
        print("8 Count  9 Display All  0 Exit")
        choice = input("Choice: ").strip()

        if choice == "1": add_student()
        elif choice == "2": search_student()
        elif choice == "3": update_student()
        elif choice == "4": delete_student()
        elif choice == "5": show_average()
        elif choice == "6": highest_scorer()
        elif choice == "7": list_by_department()
        elif choice == "8": print("Count:", len(students))
        elif choice == "9": display_all()
        elif choice == "0": print("Program ended."); break
        else: print("Invalid choice.")

if __name__ == "__main__":
    main()
```

7. Testing Report

Test	Input/Condition	Expected result
Add	New roll number	Record is added
Duplicate	Existing roll number	Duplicate message; no new record
Search	Existing roll number	Correct profile displayed
Search	Unknown roll number	Student not found
Update	Existing roll number	Selected fields updated
Delete	Existing roll number	Record removed
Average	Marks 70, 80, 90	Average = 80
Highest	Multiple students	Highest average selected
Department	Existing department	Matching students listed
Empty	No student records	Safe message; no crash
Boundary	One subject / one student	Correct result
Invalid	Unknown menu option	Invalid choice message

8. Observations

The system integrates strings, lists, tuples, sets and dictionaries in one application. Duplicate checking prevents repeated roll numbers. The program also handles missing records and empty data safely.

9. Conclusion

The Student Record and Academic Management System demonstrates how Python collections can be combined to organize academic information. It supports the required record operations and provides a structured example of problem analysis, algorithm design, pseudocode, implementation and testing.

10. Submission Checklist

■ Problem Analysis Report

■ Data Structure Selection Report

■ Algorithm Design

■ Flowchart Collection

■ Pseudocode

■ Python Source Code

■ Test Cases and Observations

■ Project Documentation / README